Workspace Compilation Studio: Deployment & Operations Manual

> **System Classification:** Asynchronous Multi-Threaded Workspace Orchestrator
>
> **Target Platforms:** Windows 10/11, macOS 12+ (Intel/Apple Silicon), Linux (Debian, Ubuntu, RHEL)
>
> **Upstream Dependency:** NotebookLM Cloud API via `notebooklm` CLI tool

1. System Architecture & Overview

The Workspace Compilation Studio is an automation suite designed to provision, seed, analyze, and structure isolated workspaces within NotebookLM. Running operations across an asynchronous thread framework, this suite enables the simultaneous deployment of independent workspaces (e.g., `Customer1` through `Customer6`).

By isolating operational memory caches, the engine runs continuous tasks—such as batch uploads, deep cloud indexing loops, and workspace asset generation blocks—without blocking the central system thread or local GUI.

1.1 Pipeline Operational Blueprints

- **`pipeline_fast.py`:** Optimized for rapid context assembly. It drops explicit mode flags when adding research files, forcing NotebookLM to run documents through its standard, high-velocity cloud indexing paths.
- **`pipeline_deep.py`:** Optimized for extensive cross-web validation cycles. It explicitly triggers deep cloud research queries and monitors active status codes. To remain within context constraints, it applies an automated truncation logic filter that imports only the top $N = 25$ cited website references as operational source assets.

2. Infrastructure Pre-Processing Lifecycle

Before launching either execution engine, source files must undergo a mandatory sanitization and data transformation loop on the local disk.

2.1 Google Drive Acquisition & Folder Topography

Source assets are maintained as compressed archive directories on Google Drive. Operators must strictly observe this data staging pipeline:

1. Download all required client asset `.zip` bundles from your Google Drive account directly onto your local workstation.
2. Create a standard project root folder named after the managing Account Executive's last name combined with the year (e.g., `Smith_2026/`).
3. Decompress and extract all downloaded customer zip structures directly into that parent folder space on your local hard drive.

2.2 Automatic Document Normalization & Sanitization

The underlying automation environment enforces strict requirements: filenames must not contain whitespace characters, and documents must be formatted as raw PDFs. To avoid manual cleanups, run the included batch utilities:

- **Format Normalization (`convert.sh`):** This automated shell wrapper launches a headless, background LibreOffice server context to recursively process your target folder, transforming raw extensions (DOCX, XLSX, PPTX) into readable PDF assets.
- **Filename Sanitization (`san.sh`):** Run this shell script to scrub your directories. It sweeps through target folders and replaces spaces with uniform underscores (`_`), preventing path-parsing errors during CLI transmission.

3. Configuration Parameter Layout (vars.py)

The multi-threaded background handlers rely on a single control properties layout named `vars.py` sitting in the root project folder. This schema maps arbitrary profile indexes cleanly to localized system folders. It strips out legacy remote cloud identifiers and contains no hardcoded customer names:

```
# vars.py - Configuration Parameters payload
clients = ["Customer1", "Customer2", "Customer3", "Customer4", "Customer5", "Customer6"]

Customer1_name = "Generic Customer One Operations LLC"
Customer1_industry = "Technology and Infrastructure Architecture"
Customer1_folder = "/AbsolutePath/To/Smith_2026/Customer1_Sanitized_PDF/"

Customer2_name = "Generic Customer Two Enterprises"
Customer2_industry = "Global Logistics and Supply Chain Coordination"
Customer2_folder = "/AbsolutePath/To/Smith_2026/Customer2_Sanitized_PDF/"

# [Customer3 through Customer6 mirror the identical pattern blocks above]
```

4. Cross-Platform Environment Setup

4.1 Debian / Ubuntu Linux Environment

```
sudo apt-get update && sudo apt-get install -y python3-dev python3-pip python3-tk
libreoffice
python3 -m venv .venv && source .venv/bin/activate
pip install google-api-python-client google-auth-oauthlib pandas openpyxl
```

4.2 macOS Environment (Intel / Apple Silicon)

```
brew install python-tk python@3.12 libreoffice
python3.12 -m venv .venv && source .venv/bin/activate
pip install google-api-python-client google-auth-oauthlib pandas openpyxl
```

4.3 Microsoft Windows Environment

```
# Install LibreOffice via official binaries, then execute in PowerShell:
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install google-api-python-client google-auth-oauthlib pandas openpyxl
```

5. Pipeline Execution Sequence

Before triggering operations, ensure a persistent cloud security handshake token is active by running `notebooklm login` in your shell. Once authenticated, run your chosen automation suite version:

```
# Launch Fast Lane Performance Tracking Engine
python pipeline_fast.py

# Launch Deep Research Resource Capping Engine
python pipeline_deep.py
```

5.1 Synchronized Refresh Monitoring Loop

Both automation architectures immediately launch an internal visualizer dashboard file named `pipeline_dashboard.html` using your default system web browser.

To ensure maximum system efficiency, this document uses a built-in JavaScript event controller. It maintains a hyper-responsive, 1-second refresh rate during the first 5 seconds to track workspace initialization steps, then scales back to a stable, 5-second auto-refresh frequency to minimize browser overhead while the multi-threaded upload loops run.